LAB 7: Fuzz Testing

Input validation for the airline baggage functions

This report follows the supplied fuzz-testing handout using the filenames in the assigned repository: `airline_baggage.py` and `test_airline_baggage.py`. It records the original run, the Hypothesis failures, the fixes, and the final passing run.

Changes made

Function	Input checks and result
`parse_booking`	Requires a 2-3 letter airline code, a positive flight number, and a positive seat row followed by one letter. Invalid values return `None`.
`total_weight`	Requires comma-separated finite, nonnegative numbers. Invalid or overflowing totals return `None`; valid weights use `math.fsum`.
`tag_label`	Requires a first and last name, supports middle names, and uses the final name in the label. Invalid values return `None`.

Test outcome

The three focused Hypothesis properties failed against the starter code. They exposed malformed booking codes, blank or non-string weight inputs, and incomplete or non-name passenger inputs. After hardening the functions, the full suite passed: **25 passed**. The suite includes 100-example Hypothesis properties, generated valid values, and regression cases.

Execution image note

The four images below are clean captures rendered from the recorded command output. The original and fixed application runs use the handout's example booking, weights, and passenger name.

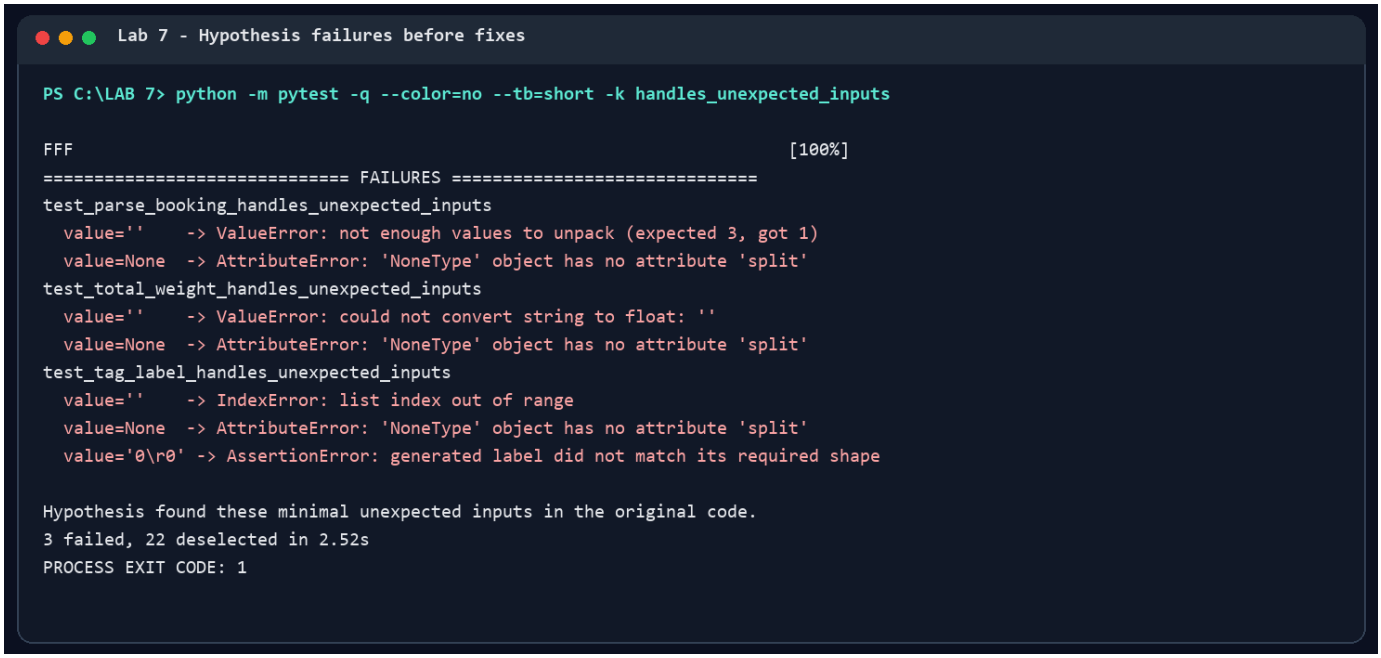
Step 2 - Run the original application

The starter application runs with valid input and prints the expected booking, weight, and tag results.



Step 4 - Fuzz failures before fixes

Hypothesis found small malformed inputs that crashed or produced an invalid passenger label.



Step 5 - Run the fixed application

The hardened application preserves the handout example outputs.



Step 6 - Rerun the tests

The property-based and regression suite passes after the fixes.

